Mini Compiler using Flex and Bison

Compiler Design Project Report

Name: Sachitha Ravichandran

Register Number: RA2311026050007

Subject: Compiler Design

Course: B.Tech CSE-AIML

1. Project Description

This project implements a simple yet functional compiler for a custom Domain-Specific Language (DSL). The compiler is developed using the C programming language along with Flex (Lex) for lexical analysis and Bison (YACC) for syntax analysis.

The objective of this project is to demonstrate the fundamental phases of a compiler in a simplified manner. The compiler processes input DSL statements and converts them into intermediate code while performing syntax and semantic validation.

Supported DSL Features

- Variable assignment
- Arithmetic operations (Addition and Subtraction)
- Multi-line statements
- Print operation

Example DSL Code

```
SET a = 5

SET b = 10

ADD a b

SUB a b

PRINT a
```

2. Tools Used

Tool	Purpose
GCC	Compiles C source code
Flex	Lexical analyzer generator
Bison	Parser generator
Ubuntu	Development environment
C Language	Implementation

3. Compiler Architecture

The compiler follows a multi-phase pipeline. Each phase transforms the input into a more structured representation.

Input DSL Code

↓

Lexical Analysis (Flex)

↓

Syntax Analysis (Bison)

↓

Semantic Analysis (Symbol Table)

↓

Intermediate Code Generation

↓

Output

Explanation

- The input DSL code is first tokenized using Flex
- The tokens are then validated using grammar rules in Bison
- Semantic analysis ensures correct usage of variables
- Finally, intermediate code is generated

4. Module Explanation

4.1 Lexical Analyzer (lexer.l)

The lexical analyzer is implemented using Flex. It scans the input DSL program and converts it into tokens such as:

- Keywords (SET, ADD, SUB, PRINT)
- Identifiers (variables)
- Numbers

Whitespace and invalid characters are handled appropriately.

4.2 Parser (parser.y)

The parser is implemented using Bison. It defines grammar rules for the DSL language and ensures that the input follows correct syntax.

Each valid statement triggers actions such as:

- Variable assignment
- Arithmetic operations
- Printing values

4.3 AST Construction

In this implementation, an explicit Abstract Syntax Tree (AST) is not separately created. Instead, parsing actions directly process statements, which acts as an implicit AST mechanism.

4.4 Semantic Analysis

Semantic analysis is performed using a symbol table.

- Stores variable values
- Ensures variables are defined before use
- Prevents invalid operations

4.5 Intermediate Code Generation

The compiler generates simple three-address-like intermediate code.

Example:

t1 = a + b

This represents arithmetic operations in a structured format.

5. Project Structure

```
CD_project/
|
 ├── src/
|    ├── lexer.l
|    ├── parser.y
|    ├── lex.yy.c
|    ├── parser.tab.c
|    ├── parser.tab.h
|    ├── compiler
|
 ├── test/
|    └── input.txt
|
 ├── output/
|    └── RA2311026050007_Sachitha_Output.png
|
 ├── docs/
|    └── Report.pdf
|
└── README.md
```

6. Steps to Run

Step 1: Install Dependencies

sudo apt update

sudo apt install gcc flex bison -y

Step 2: Build the Compiler

cd src

flex lexer.l

bison -d parser.y

gcc lex.yy.c parser.tab.c -o compiler

Step 3: Run the Compiler

./compiler < ../test/input.txt

7. Sample Input and Output

Input

SET a = 5

SET b = 10

ADD a b

SUB a b

PRINT a

Output

Assigned a = 5

Assigned b = 10

t1 = a + b

t1 = a - b

Print a = 5

8. Result

The compiler successfully processes DSL input, validates syntax and semantics, and generates correct intermediate code.

9. Conclusion

This project demonstrates the core concepts of compiler design using Flex and Bison. It covers all major phases including lexical analysis, parsing, semantic analysis, and code generation.

The implementation provides a clear understanding of how compilers work internally and serves as a strong foundation for advanced compiler development.